

Results Analysis

```
import pandas as pd
import os
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import shapiro
import scipy.stats as stats
```

Obtaining datasets

```
lvdsl_output=[
    ## Primer modelo
    "./lvdsl_outputs/VacuaFound_07052026_174039/GA/GA-0000.csv.csv",
    "./lvdsl_outputs/VacuaFound_07052026_174039/GA/GA-0001.csv.csv",
    "./lvdsl_outputs/VacuaFound_07052026_174039/GA/GA-0002.csv.csv",
    ## Segundo modelo
    "./lvdsl_outputs/VacuaFound_07052026_174049/GA/GA-0000.csv.csv",
    "./lvdsl_outputs/VacuaFound_07052026_174049/GA/GA-0001.csv.csv",
    "./lvdsl_outputs/VacuaFound_07052026_174049/GA/GA-0002.csv.csv",
    ## Tercer modelo
    "./lvdsl_outputs/VacuaFound_07052026_174804/GA/GA-0000.csv.csv",
    "./lvdsl_outputs/VacuaFound_07052026_174804/GA/GA-0001.csv.csv",
    "./lvdsl_outputs/VacuaFound_07052026_174804/GA/GA-0002.csv.csv",
    ## Cuarto modelo
    "./lvdsl_outputs/VacuaFound_07052026_175255/GA/GA-0000.csv.csv",
    "./lvdsl_outputs/VacuaFound_07052026_175255/GA/GA-0001.csv.csv",
    "./lvdsl_outputs/VacuaFound_07052026_175255/GA/GA-0002.csv.csv",
]

lvdsl_runmodels=[
    ## Primer modelo
    "./lvdsl_outputs/VacuaFound_07052026_174039/GA/model_description.txt",
```

```

## Segundo modelo
"./lvdsl_outputs/VacuaFound_07052026_174049/GA/model_description.txt",
## Tercer modelo
"./lvdsl_outputs/VacuaFound_07052026_174804/GA/model_description.txt",
## Cuarto modelo
"./lvdsl_outputs/VacuaFound_07052026_175255/GA/model_description.txt",
]

data=[]

for dataset in lvdsl_output:
    data.append(pd.read_csv(dataset))

```

by Model grouping

```

i=0
for mod in lvdsl_runmodels:
    i=i+1
    print(f'Model {i}')
    with open(mod) as f:
        print(f.readlines())

```

```

Model 1
['BaseGA(epoch=50, pop_size=20, pc=0.1, pm=0.2)\n']
Model 2
['BaseGA(epoch=100, pop_size=60, pc=0.9, pm=0.8)\n']
Model 3
['BaseGA(epoch=50, pop_size=20, pc=0.5, pm=0.6)\n']
Model 4
['BaseGA(epoch=50, pop_size=60, pc=0.5, pm=0.6)\n']

```

```

all_frames = []

for i, path in enumerate(lvdsl_output):
    df = pd.read_csv(path)
    # Assign model group (0-2 is Model 1, 3-5 is Model 2, etc.)
    df['Model'] = f'Model {i // 3 + 1}'
    # Assign Run ID (0, 1, or 2 within that model)
    df['Run'] = f'Run {i % 3}'
    all_frames.append(df[['V', 'fitness', 's', 'tau', 'Model', 'Run']])

```

Run characteristics

```
for frame in all_frames[0:3]:
    run=frame['Run'].iloc[0]
    run=int(run[3:])+1
    s=frame['s'].iloc[0]
    tau=frame['tau'].iloc[0]
    print(f'Run {run}: s={s}, tau={tau}')
```

```
Run 1: s=1.600149459561681, tau=3.485910062937601
Run 2: s=2.459242111292193, tau=3.0947272010640288
Run 3: s=1.053781742958659, tau=1.4533862399266009
```

Stable vacua count

```
# 1. Prepare the Data
plot_list = []
for i, ds in enumerate(data):
    model_num = i // 3
    run_idx = i % 3 # 0, 1, or 2
    stable_count = ds[ds['taquiónico'] == 0].shape[0]

    # Mapping indices to names for the plot
    run_names = ['Run 1', 'Run 2', 'Run 3']

    plot_list.append({
        'Model': f'Model {model_num + 1}',
        'Run': run_names[run_idx],
        'Stable_Count': stable_count,
        'Total_Count': ds.shape[0]
    })

df_plot = pd.DataFrame(plot_list)

# 2. Define the Autumn Forest Palette (Per Run)
# Forest Green, Burnt Orange, Earthy Brown
autumn_palette = ["#228B22", "#D2691E", "#8B4513"]

# 3. Plotting
plt.figure(figsize=(12, 7))

ax = sns.barplot(
```

```

    data=df_plot,
    x='Model',
    y='Stable_Count',
    hue='Run',
    palette=autumn_palette,
    edgecolor='black'
)

# Aesthetics
plt.title('Stability Analysis: Count of Non-Tachyonic Vacua', fontsize=14, fontweight='bold')
plt.ylabel('Number of Stable (Non Tachionic) Points', fontsize=12)
plt.xlabel('Model Comparison', fontsize=12)
plt.ylim(0, 105) # Since you have 100 rows per dataset
plt.grid(axis='y', linestyle='--', alpha=0.3)

# Legend placement
plt.legend(title="Execution Runs", loc='upper left', bbox_to_anchor=(1, 1))

plt.tight_layout()
plt.savefig('StabilityAnalysis.eps')
plt.show()

```

/usr/lib/python3.14/site-packages/seaborn/_base.py:1447: FutureWarning: ChainedAssignmentError: You are setting values through chained assignment. Currently this works in certain cases, but on-Write (which will become the default behaviour in pandas 3.0) this will never work to update. A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
self.plot_data[axis] = cat_data
```

/usr/lib/python3.14/site-packages/seaborn/categorical.py:403: FutureWarning: ChainedAssignmentError: You are setting values through chained assignment. Currently this works in certain cases, but on-Write (which will become the default behaviour in pandas 3.0) this will never work to update. A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
data["width"] /= n
```

/usr/lib/python3.14/site-packages/seaborn/categorical.py:407: FutureWarning: ChainedAssignmentError: You are setting values through chained assignment. Currently this works in certain cases, but on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the DataFrames. A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
data[self.orient] += offset
```

/usr/lib/python3.14/site-packages/seaborn/categorical.py:1292: FutureWarning: ChainedAssignmentError: You are setting values through chained assignment. Currently this works in certain cases, but on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the DataFrames. A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
agg_data["edge"] = agg_data[self.orient] - agg_data["width"] / 2
```

/usr/lib/python3.14/site-packages/seaborn/categorical.py:415: FutureWarning: ChainedAssignmentError: You are setting values through chained assignment. Currently this works in certain cases, but on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the DataFrames. A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
data["edge"] = inv(data[var] - hw)
```

/usr/lib/python3.14/site-packages/seaborn/categorical.py:416: FutureWarning: ChainedAssignmentError: You are setting values through chained assignment. Currently this works in certain cases, but in the future it will not. To prevent this, use df.loc[row_indexer, "col"] = values instead, to perform the assignment in a single step. (on Write (which will become the default behaviour in pandas 3.0) this will never work to update the DataFrame. A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
data["width"] = inv(data[var] + hw) - data["edge"].to_numpy()
```

/usr/lib/python3.14/site-packages/seaborn/categorical.py:419: FutureWarning: ChainedAssignmentError: You are setting values through chained assignment. Currently this works in certain cases, but in the future it will not. To prevent this, use df.loc[row_indexer, "col"] = values instead, to perform the assignment in a single step. (on Write (which will become the default behaviour in pandas 3.0) this will never work to update the DataFrame. A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
data[col] = inv(data[col])
```

/usr/lib/python3.14/site-packages/seaborn/categorical.py:403: FutureWarning: ChainedAssignmentError: You are setting values through chained assignment. Currently this works in certain cases, but in the future it will not. To prevent this, use df.loc[row_indexer, "col"] = values instead, to perform the assignment in a single step. (on Write (which will become the default behaviour in pandas 3.0) this will never work to update the DataFrame. A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
data["width"] /= n
```

/usr/lib/python3.14/site-packages/seaborn/categorical.py:407: FutureWarning: ChainedAssignmentError: You are setting values through chained assignment. Currently this works in certain cases, but in the future it will not. To prevent this, use df.loc[row_indexer, "col"] = values instead, to perform the assignment in a single step.

on-Write (which will become the default behaviour in pandas 3.0) this will never work to update. A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
data[self.orient] += offset
```

/usr/lib/python3.14/site-packages/seaborn/categorical.py:1292: FutureWarning: ChainedAssignmentError: You are setting values through chained assignment. Currently this works in certain cases, but on-Write (which will become the default behaviour in pandas 3.0) this will never work to update. A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
agg_data["edge"] = agg_data[self.orient] - agg_data["width"] / 2
```

/usr/lib/python3.14/site-packages/seaborn/categorical.py:415: FutureWarning: ChainedAssignmentError: You are setting values through chained assignment. Currently this works in certain cases, but on-Write (which will become the default behaviour in pandas 3.0) this will never work to update. A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
data["edge"] = inv(data[var] - hw)
```

/usr/lib/python3.14/site-packages/seaborn/categorical.py:416: FutureWarning: ChainedAssignmentError: You are setting values through chained assignment. Currently this works in certain cases, but on-Write (which will become the default behaviour in pandas 3.0) this will never work to update. A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
data["width"] = inv(data[var] + hw) - data["edge"].to_numpy()
/usr/lib/python3.14/site-packages/seaborn/categorical.py:419: FutureWarning: ChainedAssignmentError:
You are setting values through chained assignment. Currently this works in certain cases, but
on-Write (which will become the default behaviour in pandas 3.0) this will never work to update
A typical example is when you are setting values in a column of a DataFrame, like:
```

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
data[col] = inv(data[col])
/usr/lib/python3.14/site-packages/seaborn/categorical.py:403: FutureWarning: ChainedAssignmentError:
You are setting values through chained assignment. Currently this works in certain cases, but
on-Write (which will become the default behaviour in pandas 3.0) this will never work to update
A typical example is when you are setting values in a column of a DataFrame, like:
```

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
data["width"] /= n
/usr/lib/python3.14/site-packages/seaborn/categorical.py:407: FutureWarning: ChainedAssignmentError:
You are setting values through chained assignment. Currently this works in certain cases, but
on-Write (which will become the default behaviour in pandas 3.0) this will never work to update
A typical example is when you are setting values in a column of a DataFrame, like:
```

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

[docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy](https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)

```
data[self.orient] += offset
```

/usr/lib/python3.14/site-packages/seaborn/categorical.py:1292: FutureWarning: ChainedAssignmentError: You are setting values through chained assignment. Currently this works in certain cases, but on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the DataFrame. A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
agg_data["edge"] = agg_data[self.orient] - agg_data["width"] / 2
```

/usr/lib/python3.14/site-packages/seaborn/categorical.py:415: FutureWarning: ChainedAssignmentError: You are setting values through chained assignment. Currently this works in certain cases, but on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the DataFrame. A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
data["edge"] = inv(data[var] - hw)
```

/usr/lib/python3.14/site-packages/seaborn/categorical.py:416: FutureWarning: ChainedAssignmentError: You are setting values through chained assignment. Currently this works in certain cases, but on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the DataFrame. A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
data["width"] = inv(data[var] + hw) - data["edge"].to_numpy()
```

/usr/lib/python3.14/site-packages/seaborn/categorical.py:419: FutureWarning: ChainedAssignmentError: You are setting values through chained assignment. Currently this works in certain cases, but on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the DataFrame. A typical example is when you are setting values in a column of a DataFrame, like:

You are setting values through chained assignment. Currently this works in certain cases, but on-Write (which will become the default behaviour in pandas 3.0) this will never work to update. A typical example is when you are setting values in a column of a DataFrame, like:

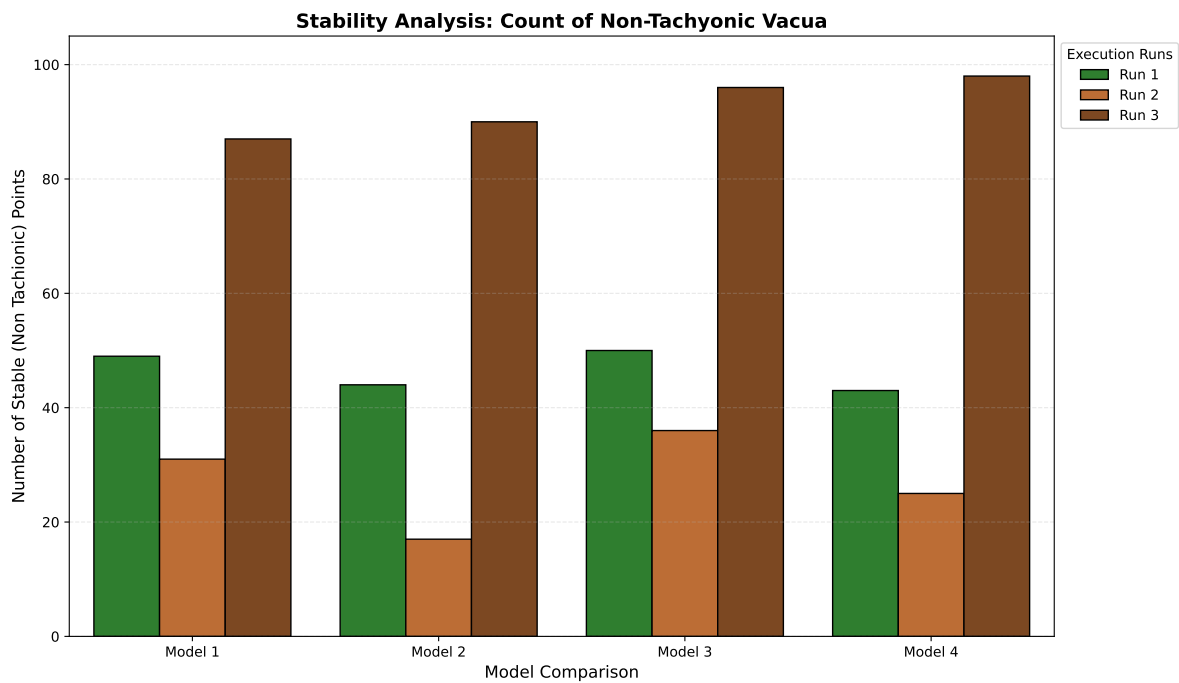
```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
data[col] = inv(data[col])
```

The PostScript backend does not support transparency; partially transparent artists will be rendered as solid.



Stability ratio

```
for e in plot_list:
    print(f"Model {e['Model']} Run {e['Run']} Stability Ratio: {e['Stable_Count']/e['Total_Count']}")
```

```
Model Model 1 Run Run 1 Stability Ratio: 0.49
Model Model 1 Run Run 2 Stability Ratio: 0.31
Model Model 1 Run Run 3 Stability Ratio: 0.87
```

```

Model Model 2 Run Run 1 Stability Ratio: 0.44
Model Model 2 Run Run 2 Stability Ratio: 0.17
Model Model 2 Run Run 3 Stability Ratio: 0.99
Model Model 3 Run Run 1 Stability Ratio: 0.50
Model Model 3 Run Run 2 Stability Ratio: 0.36
Model Model 3 Run Run 3 Stability Ratio: 0.96
Model Model 4 Run Run 1 Stability Ratio: 0.43
Model Model 4 Run Run 2 Stability Ratio: 0.25
Model Model 4 Run Run 3 Stability Ratio: 0.98

```

Fitness and Potential comparison

```

sns.set_theme(style="whitegrid", context="paper", font_scale=2.0)
# Combine into one master dataframe
combined_df = pd.concat(all_frames)

# 2. Create the Comparison Plot
fig, axs = plt.subplots(2, 1, figsize=(14, 10))

# Plot for V
sns.violinplot(data=combined_df, x='Model', y='V', hue='Run', ax=axs[0], palette='viridis')
axs[0].set_title('Comparison of V across Models (3 Runs each)')
axs[0].grid(axis='y', linestyle='--', alpha=0.7)

# Plot for fitness
sns.violinplot(data=combined_df, x='Model', y='fitness', hue='Run', ax=axs[1], palette='magma')
axs[1].set_title('Comparison of Fitness across Models (3 Runs each)')
axs[1].grid(axis='y', linestyle='--', alpha=0.7)

plt.tight_layout()
plt.savefig('ViolinAG_comparison.eps')
plt.show()

```

/usr/lib/python3.14/site-packages/seaborn/_base.py:1447: FutureWarning: ChainedAssignmentError: You are setting values through chained assignment. Currently this works in certain cases, but on Write (which will become the default behaviour in pandas 3.0) this will never work to update the data. A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row\_indexer, "col"] = values` instead, to perform the assignment in a single step.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
self.plot_data[axis] = cat_data
```

/usr/lib/python3.14/site-packages/seaborn/categorical.py:403: FutureWarning: ChainedAssignmentError: You are setting values through chained assignment. Currently this works in certain cases, but on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the DataFrame. A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
data["width"] /= n
```

/usr/lib/python3.14/site-packages/seaborn/categorical.py:407: FutureWarning: ChainedAssignmentError: You are setting values through chained assignment. Currently this works in certain cases, but on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the DataFrame. A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
data[self.orient] += offset
```

/usr/lib/python3.14/site-packages/seaborn/_base.py:1447: FutureWarning: ChainedAssignmentError: You are setting values through chained assignment. Currently this works in certain cases, but on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the DataFrame. A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
self.plot_data[axis] = cat_data
```

/usr/lib/python3.14/site-packages/seaborn/categorical.py:403: FutureWarning: ChainedAssignmentError: You are setting values through chained assignment. Currently this works in certain cases, but in the future it will not work: on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the DataFrame in place. A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
data["width"] /= n
```

/usr/lib/python3.14/site-packages/seaborn/categorical.py:407: FutureWarning: ChainedAssignmentError: You are setting values through chained assignment. Currently this works in certain cases, but in the future it will not work: on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the DataFrame in place. A typical example is when you are setting values in a column of a DataFrame, like:

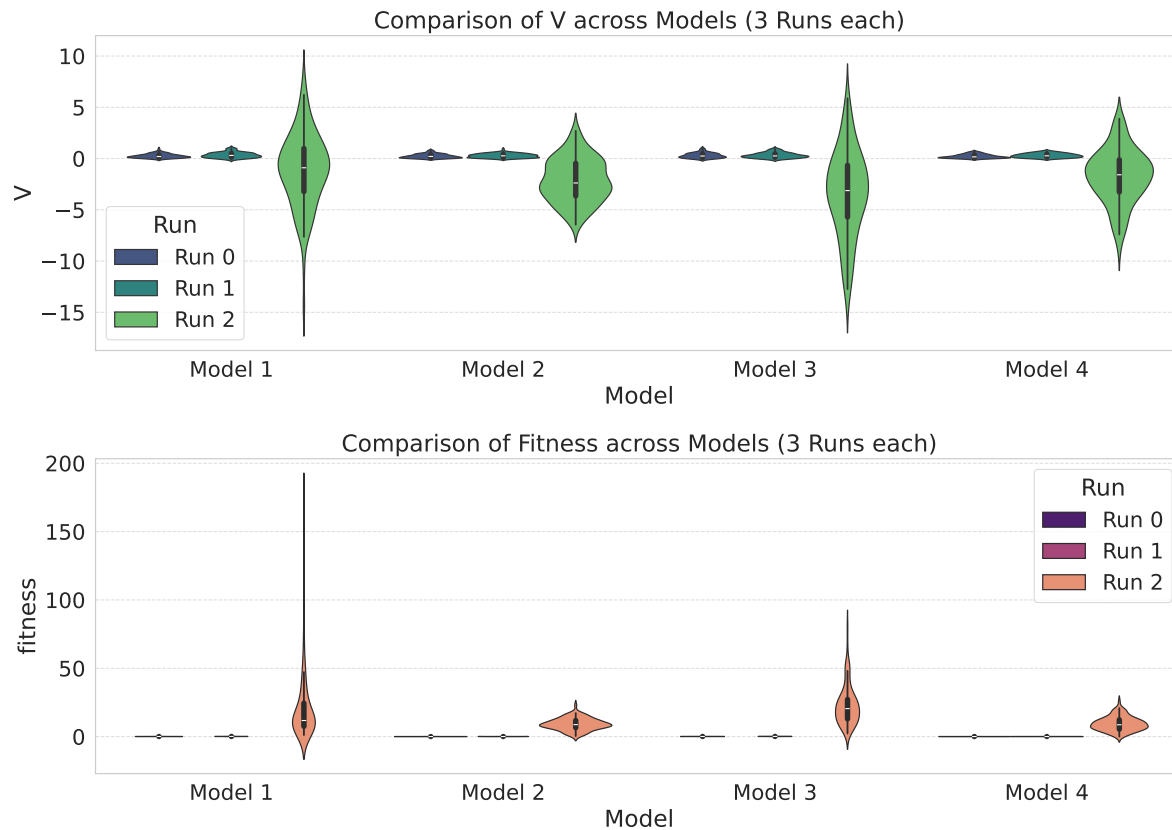
```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
data[self.orient] += offset
```

The PostScript backend does not support transparency; partially transparent artists will be rendered as opaque.



Normality test

```
normality_results = []

for i, df in enumerate(data):
    # Test for 'V'
    stat_v, p_v = shapiro(df['V'])
    # Test for 'fitness'
    stat_f, p_f = shapiro(df['fitness'])

    normality_results.append({
        'Dataset': f'DF {i}',
        'V_p-value': round(p_v, 4),
        'V_is_Normal': p_v > 0.05,
        'Fit_p-value': round(p_f, 4),
        'Fit_is_Normal': p_f > 0.05
    })

# Display as a Table
```

```

results_df = pd.DataFrame(normality_results)

#plt.figure(figsize=(6, 6))
#stats.probplot(data[0]['fitness'], dist="norm", plot=plt)
#plt.title("Q-Q Plot for Fitness (Dataset 0)")
#plt.show()

with open('AG_normality_results.csv', 'w') as f:
    f.write(results_df.to_csv())
print(results_df)

```

	Dataset	V_p-value	V_is_Normal	Fit_p-value	Fit_is_Normal
0	DF 0	0.0001	False	0.1046	True
1	DF 1	0.0007	False	0.0000	False
2	DF 2	0.1623	True	0.0000	False
3	DF 3	0.0000	False	0.3620	True
4	DF 4	0.0036	False	0.0719	True
5	DF 5	0.3750	True	0.3386	True
6	DF 6	0.0001	False	0.5932	True
7	DF 7	0.0003	False	0.0248	False
8	DF 8	0.3651	True	0.0000	False
9	DF 9	0.0000	False	0.6264	True
10	DF 10	0.0042	False	0.2522	True
11	DF 11	0.3894	True	0.0284	False

Tests

```

from scipy.stats import shapiro, f_oneway, kruskal

# Defining your matrix groups
groups = [
    [0, 3, 6, 9],
    [1, 4, 7, 10],
    [2, 5, 8, 11]
]

target_metrics = ['V', 'fitness']

for row_idx, group in enumerate(groups):
    # Extract s and tau from the first dataframe in the group
    s_val = data[group[0]]['s'].iloc[0]

```

```

tau_val = data[group[0]]['tau'].iloc[0]

print(f"--- Testing Moduli Group {row_idx}: s={s_val}, tau={tau_val} ---")

for metric in target_metrics:
    # Collect the metric arrays for the 4 dataframes in this group
    samples = [data[i][metric] for i in group]

    # 1. Normality Check (All samples in group must be normal for ANOVA)
    all_normal = all(shapiro(s)[1] > 0.05 for s in samples)

    # 2. Perform Test
    if all_normal:
        stat, p_value = f_oneway(*samples)
        test_name = "ANOVA"
    else:
        stat, p_value = kruskal(*samples)
        test_name = "Kruskal-Wallis"

    result = "Significant Difference" if p_value < 0.05 else "No Significant Difference"

    print(f" [{metric}] {test_name}: p={p_value:.4f} -> {result}")
print("\n")

```

```

--- Testing Moduli Group 0: s=1.600149459561681, tau=3.485910062937601 ---
[V] Kruskal-Wallis: p=0.1795 -> No Significant Difference
[fitness] ANOVA: p=0.0000 -> Significant Difference

```

```

--- Testing Moduli Group 1: s=2.459242111292193, tau=3.0947272010640288 ---
[V] Kruskal-Wallis: p=0.5771 -> No Significant Difference
[fitness] Kruskal-Wallis: p=0.0000 -> Significant Difference

```

```

--- Testing Moduli Group 2: s=1.053781742958659, tau=1.4533862399266009 ---
[V] ANOVA: p=0.0000 -> Significant Difference
[fitness] Kruskal-Wallis: p=0.0000 -> Significant Difference

```

Posthoc


```

from scipy.stats import mannwhitneyu

# Your matrix again
groups = [[0, 3, 6, 9], [1, 4, 7, 10], [2, 5, 8, 11]]
model_names = ["Model 1", "Model 2", "Model 3", "Model 4"]

for row_idx, group_indices in enumerate(groups):
    print(f"--- ANALYZING MODULI GROUP {row_idx} ---")

    # 1. Calculate Medians to find the 'Candidate' for best
    medians = [data[i]['fitness'].median() for i in group_indices]
    best_idx_in_group = medians.index(max(medians))
    best_model_name = model_names[best_idx_in_group]
    best_data = data[group_indices[best_idx_in_group]]['fitness']

    print(f"Candidate for Best: {best_model_name} (Median Fitness: {max(medians):.4f})")

    # 2. Pairwise Comparison (Best vs Everyone Else)
    # Since we do 3 comparisons per group, alpha becomes 0.05 / 3 = 0.0166 (Bonferroni)
    alpha_corrected = 0.05 / 3
    is_truly_superior = True

    for i, model_idx in enumerate(group_indices):
        if i == best_idx_in_group:
            continue

        compare_data = data[model_idx]['fitness']
        stat, p = mannwhitneyu(best_data, compare_data, alternative='greater')

        if p > alpha_corrected:
            print(f" - No statistical lead over {model_names[i]} (p={p:.4f})")
            is_truly_superior = False
        else:
            print(f" - Statistically better than {model_names[i]} (p={p:.4f})")

    if is_truly_superior:
        print(f"RESULT: {best_model_name} is the CLEAR WINNER for this group.\n")
    else:
        print(f"RESULT: Tie or negligible difference between top models.\n")

```

```

--- ANALYZING MODULI GROUP 0 ---
Candidate for Best: Model 3 (Median Fitness: 0.1215)

```

- Statistically better than Model 1 ($p=0.0044$)
- Statistically better than Model 2 ($p=0.0000$)
- Statistically better than Model 4 ($p=0.0000$)

RESULT: Model 3 is the CLEAR WINNER for this group.

--- ANALYZING MODULI GROUP 1 ---

Candidate for Best: Model 3 (Median Fitness: 0.1560)

- No statistical lead over Model 1 ($p=0.0552$)
- Statistically better than Model 2 ($p=0.0000$)
- Statistically better than Model 4 ($p=0.0000$)

RESULT: Tie or negligible difference between top models.

--- ANALYZING MODULI GROUP 2 ---

Candidate for Best: Model 3 (Median Fitness: 20.4978)

- Statistically better than Model 1 ($p=0.0002$)
- Statistically better than Model 2 ($p=0.0000$)
- Statistically better than Model 4 ($p=0.0000$)

RESULT: Model 3 is the CLEAR WINNER for this group.